TEL200 Introduction to Robotics

Chapter 5: Navigation/Planning

David A. Anisi

Navigation/Planning

- Reactive (behavior-based) vs. Deliberate (map-based) planning

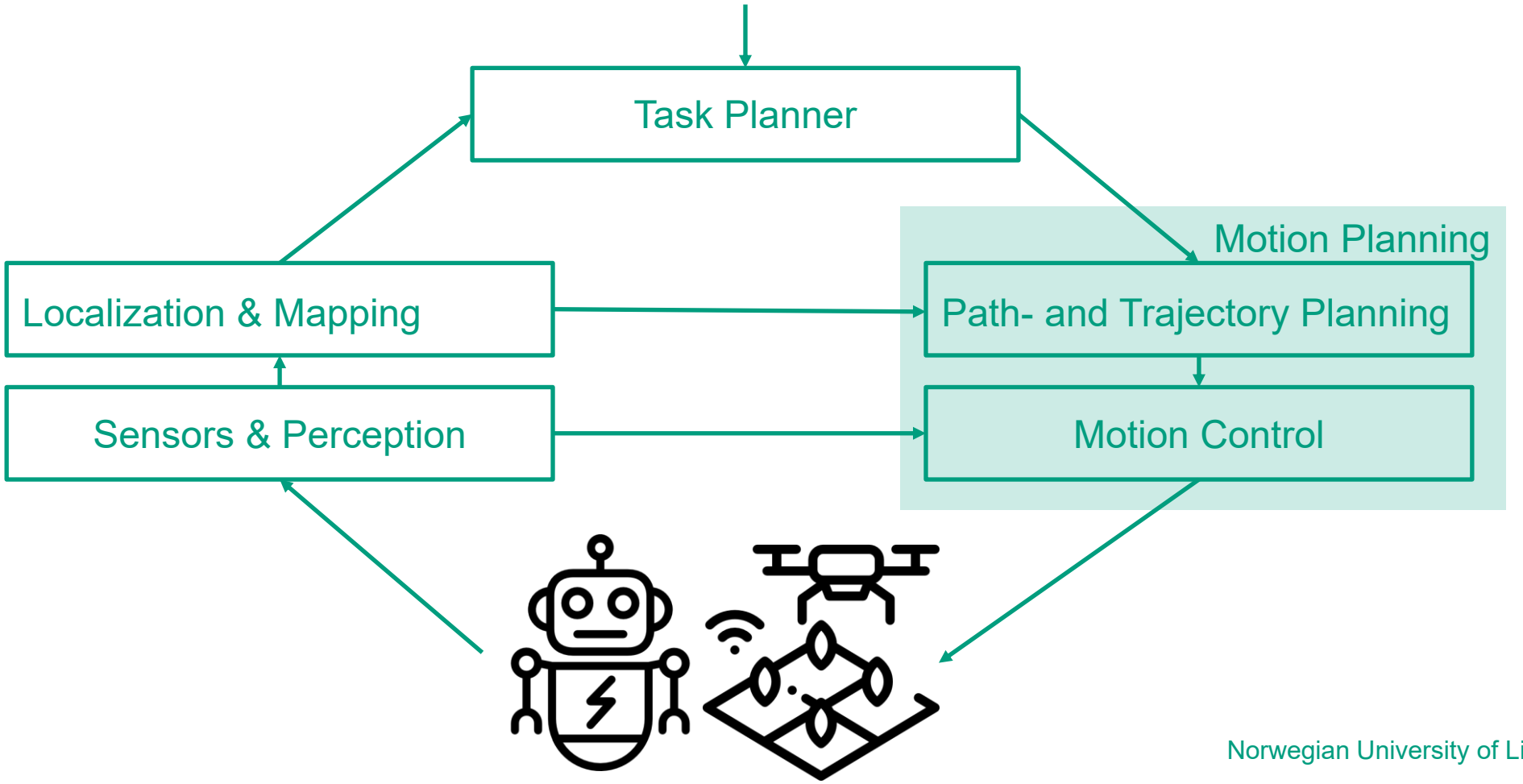
<u>Reactive (behavior-based)</u>

- Subsumption architecture
- Braitenberg vehicles
  - Phototaxis
  - Gas localization (infotaxis)
- Simple automata: bug algorithms

<u>Deliberate (map-based)</u>

- Mathematical world-representation
  - Occupancy grids/ Graphs/ Voronoi diagrams/ Octrees
- Dijkstra's, A*, D*
- Discrete Transform
- Roadmaps, PRM
- Dubins, Reed-Shepp's, Lattice Planner
- RRT, RRT*

Robotic System Overview

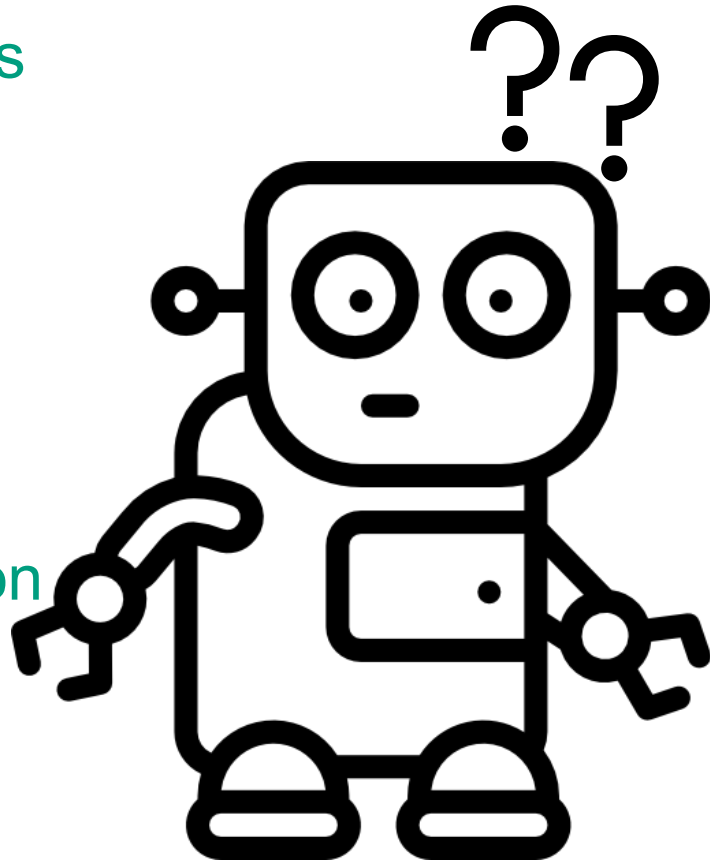


Fundamental questions in Robotics

- Where should I go? What to do there?
  - Task planning
- Where am I?
  - Localization
- What is the surrounding environment?
  - Mapping
- How do I move safely from A to B?
  - Motion planning

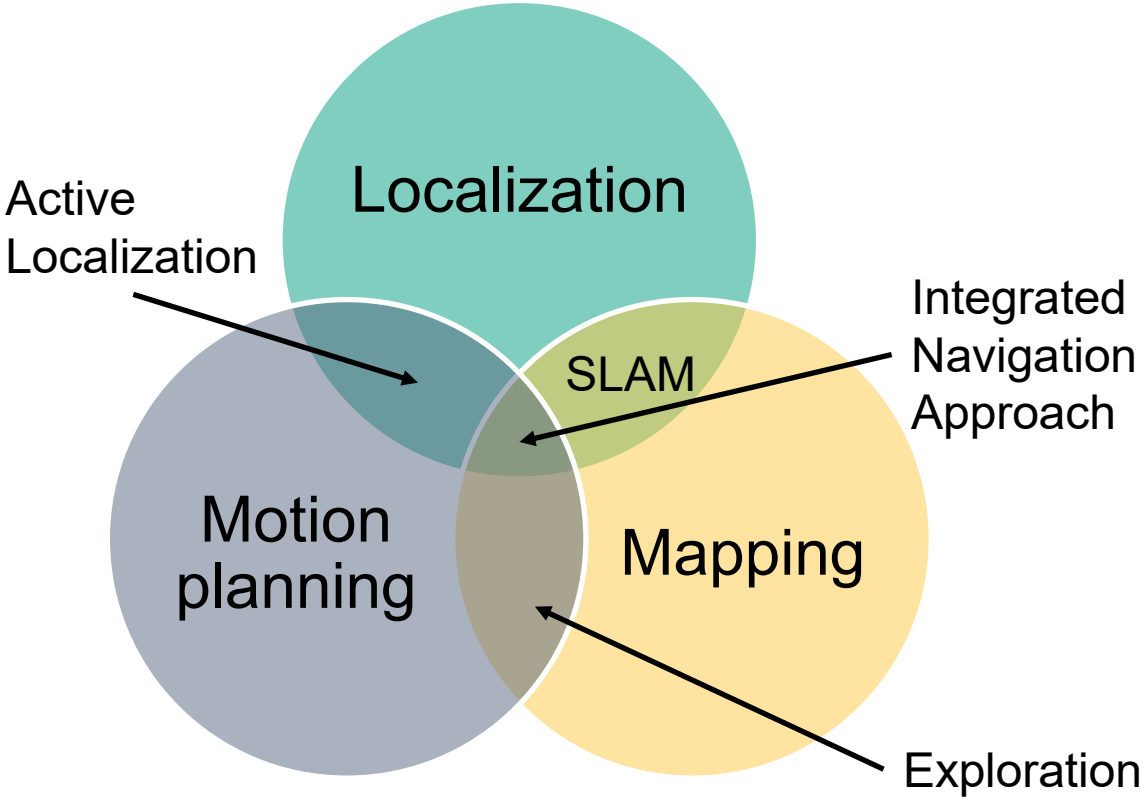
Not in this course

Robot Navigation



Robot Navigation

- «*the process of directing a vehicle so as to reach the intended destination*”
  - IEEE Standard 172-1983



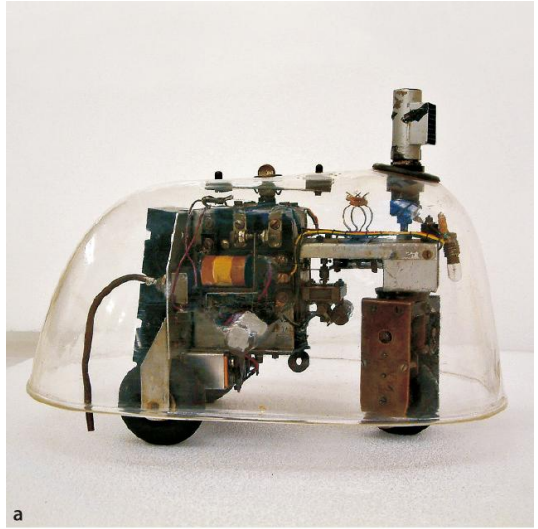
Motion Planning

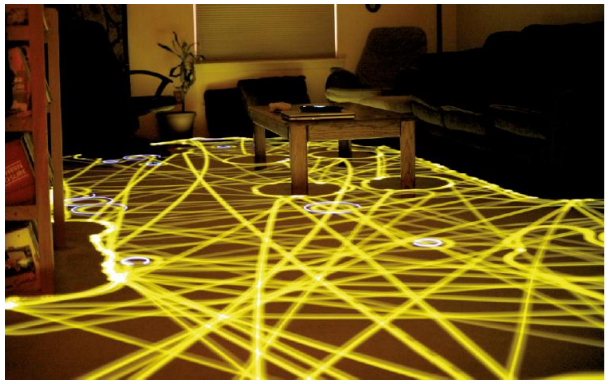
- When does a robot need to do planning?
  - Move from A to B
  - Grasp object, O
  - Assemble an object
  - Avoid unseen event (e.g. collision)
- Global planning (route optimization) vs. Local planning
  - Deliberate vs. Reactive
- In general: (path) planning is hard! Complete algorithms may take exponential time
- Have to settle for ”satisficing” (good enough) algorithms and solutions

Planning

Reactive (behavior-based)

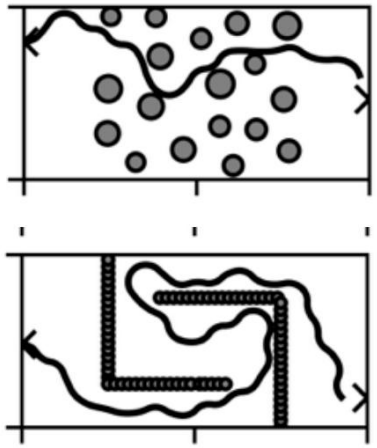
- Elise the turtoise (1948): seek out light source (Phototaxis)





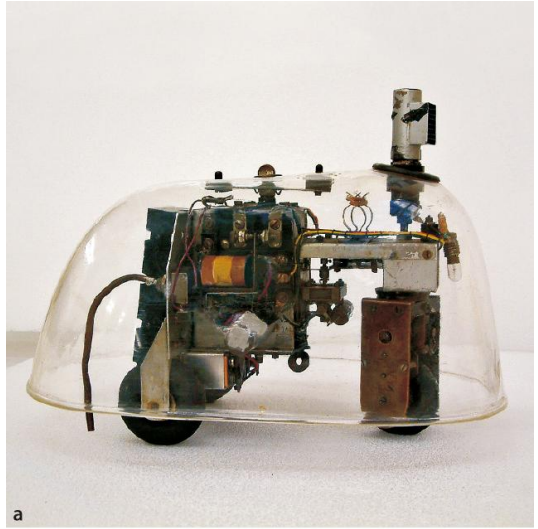
Deliberate (map-based)

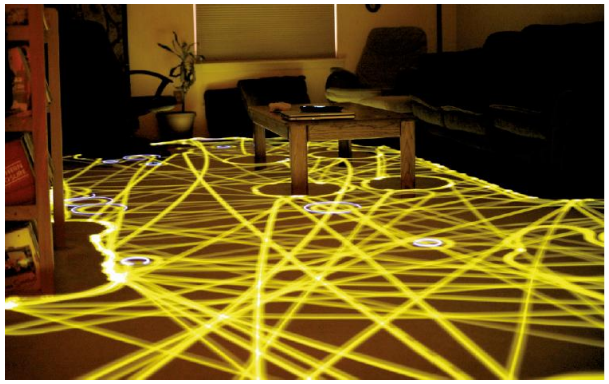
- Shakey (1968) : 3D perception (camera, laser), grid world-model, A* (graph-based) planning

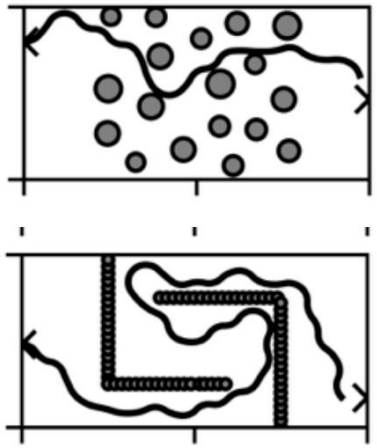


Student group activity: Reactive- vs. Map-based planning

- Discuss and note some pros/cons with the two approaches.
- In which situations/applications are they particularly useful?







Planning: ”intelligence is in the eyes of the observer”

Reactive (behavior-based)

- No explicit world representation
  - "the world is its own best model"
- No explicit plan
- No need for localization
- Fast (real-time) planning
- Complexity scaling limitations
- Simple organisms (insects)

Deliberate (map-based)

- World-map needed
- Explicit (deliberate) planning
- Localization needed
- High computational load and representation cost
- Capable of more complex tasks
- Resemble human-like capabilities
- Complex organisms (mammals)

Reactive / Behavior-based

- Subsumption architecture, 1985

MASSACHUSETTS INSTITUTE OF TECHNOLOGY
ARTIFICIAL INTELLIGENCE LABORATORY

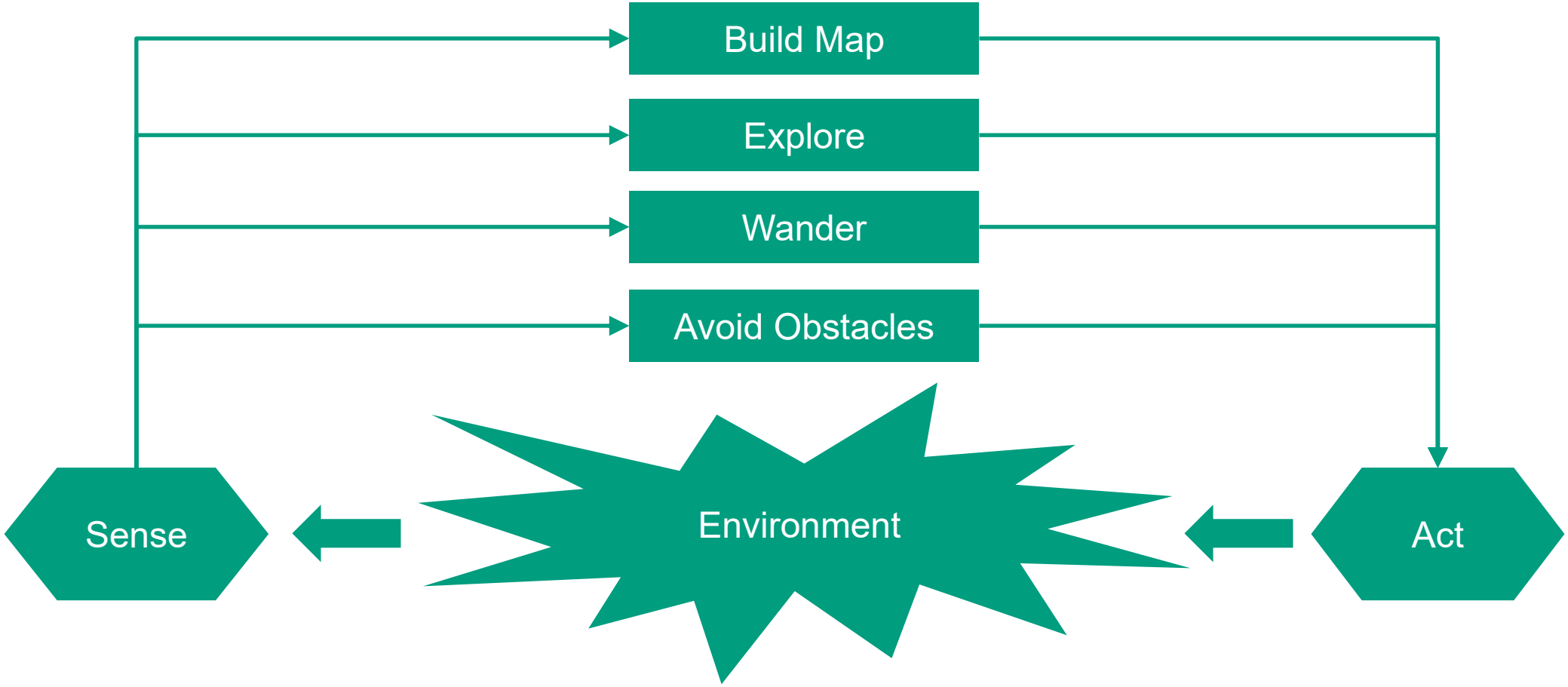
A. I. Memo 864 **September, 1985**

A ROBUST LAYERED CONTROL SYSTEM FOR A MOBILE ROBOT

Rodney A. Brooks

Abstract. We describe a new architecture for controlling mobile robots. Layers of control system are built to let the robot operate at increasing levels of competence. Layers are made up of asynchronous modules which communicate over low bandwidth channels. Each module is an instance of a fairly simple computational machine. Higher level layers can subsume the roles of lower levels by suppressing their outputs. However, lower levels continue to function as higher levels are added. The result is a robust and flexible robot control system. The system is intended to control a robot that wanders the office areas of our laboratory, building maps of its surroundings. In this paper we demonstrate the system controlling a detailed simulation of the robot.

Reactive / Behavior-based

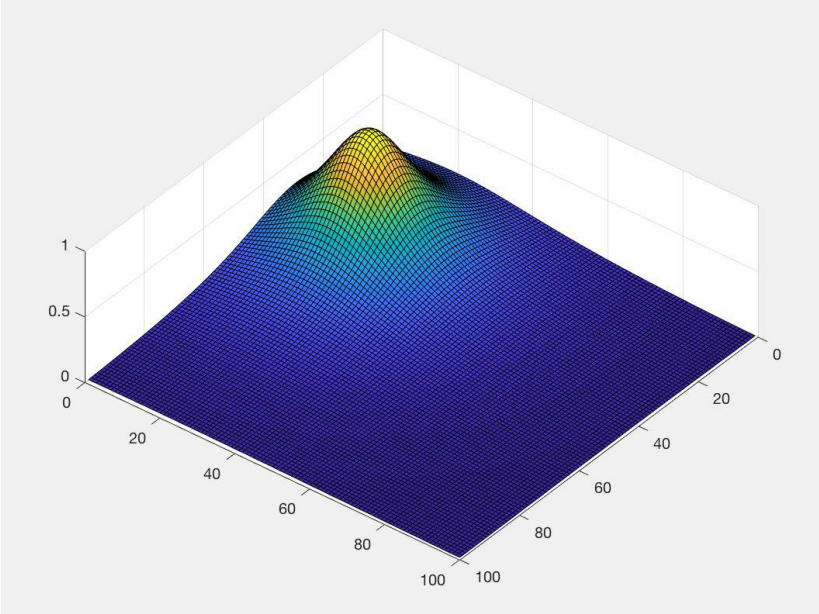


Rective planning – Braitenberg vehicles

- Seek (local) maxima of a scalar field (light intensity/chemical concentration) in 2D
- Using bilateral sensing (one sensor on each side)

Sensor field: $sf(x,y) = \frac{200}{(x-xc)^2+(y-yc)^2+200}$

Sensor reading: $s(x,y) \in [0,1]$

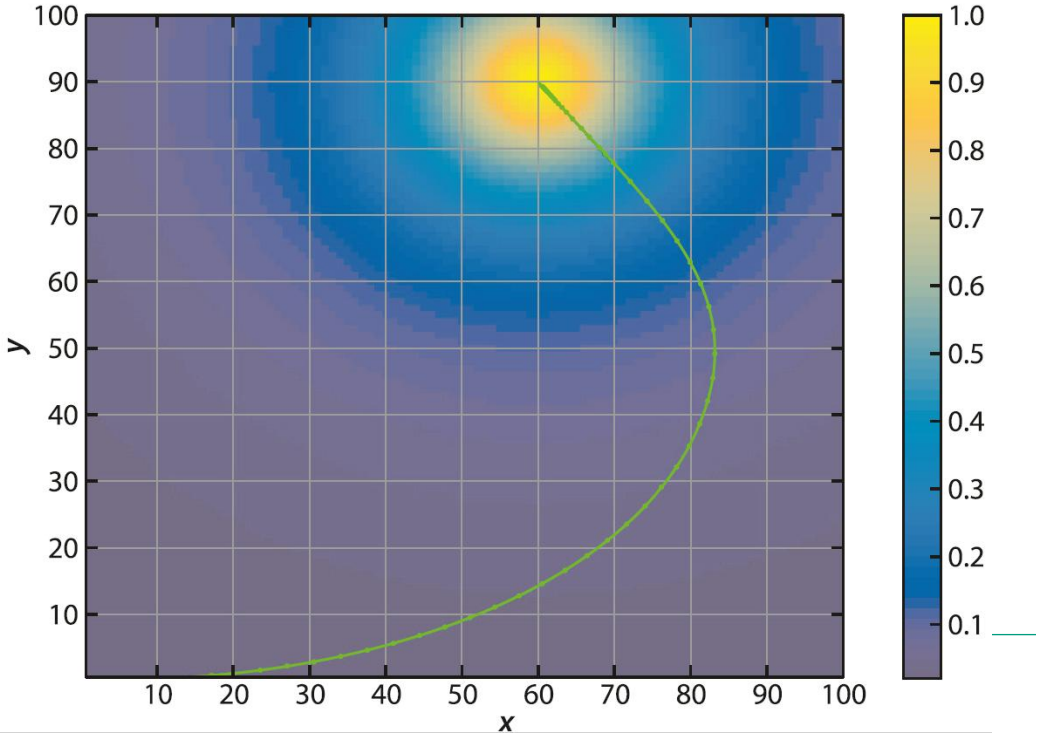


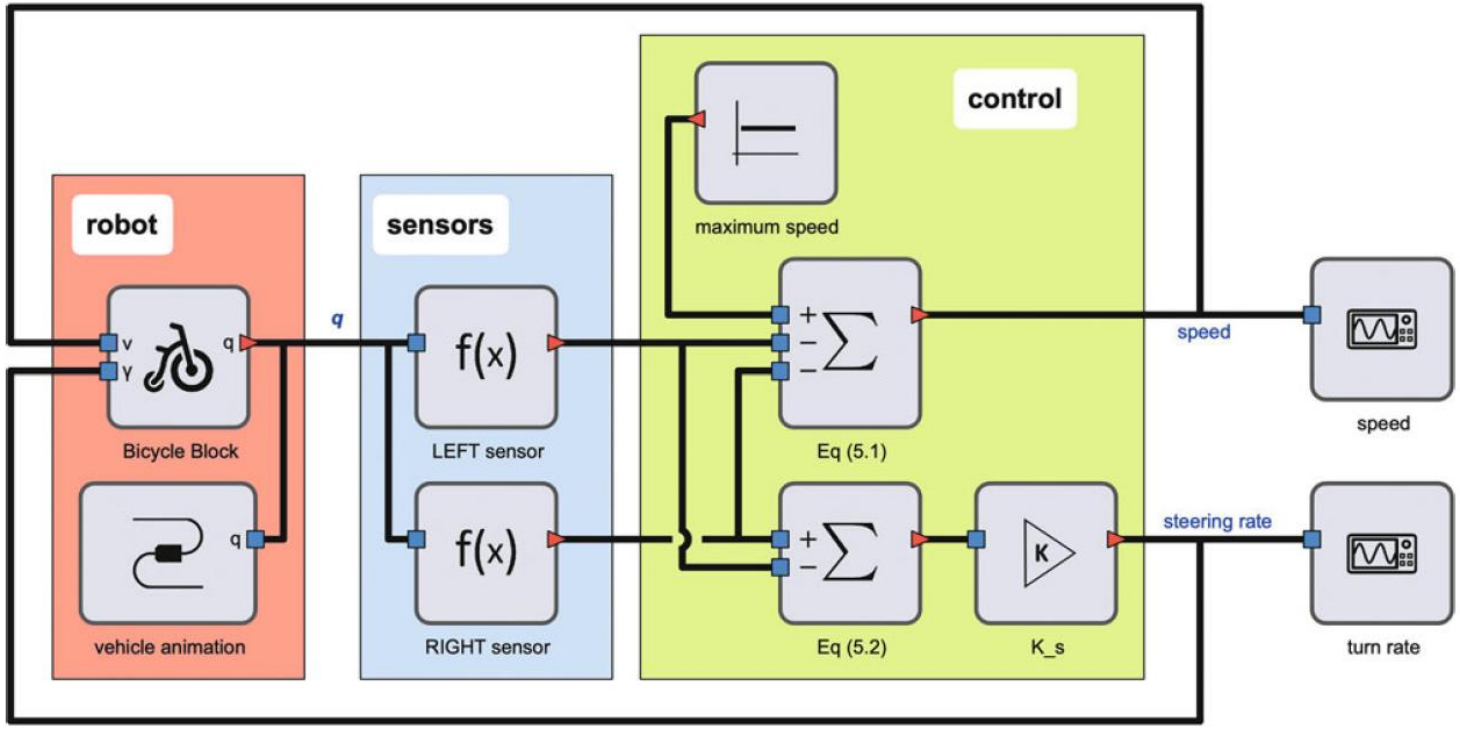
Rective planning – Braitenberg vehicles

- Change conf. in models/braitenberg.py

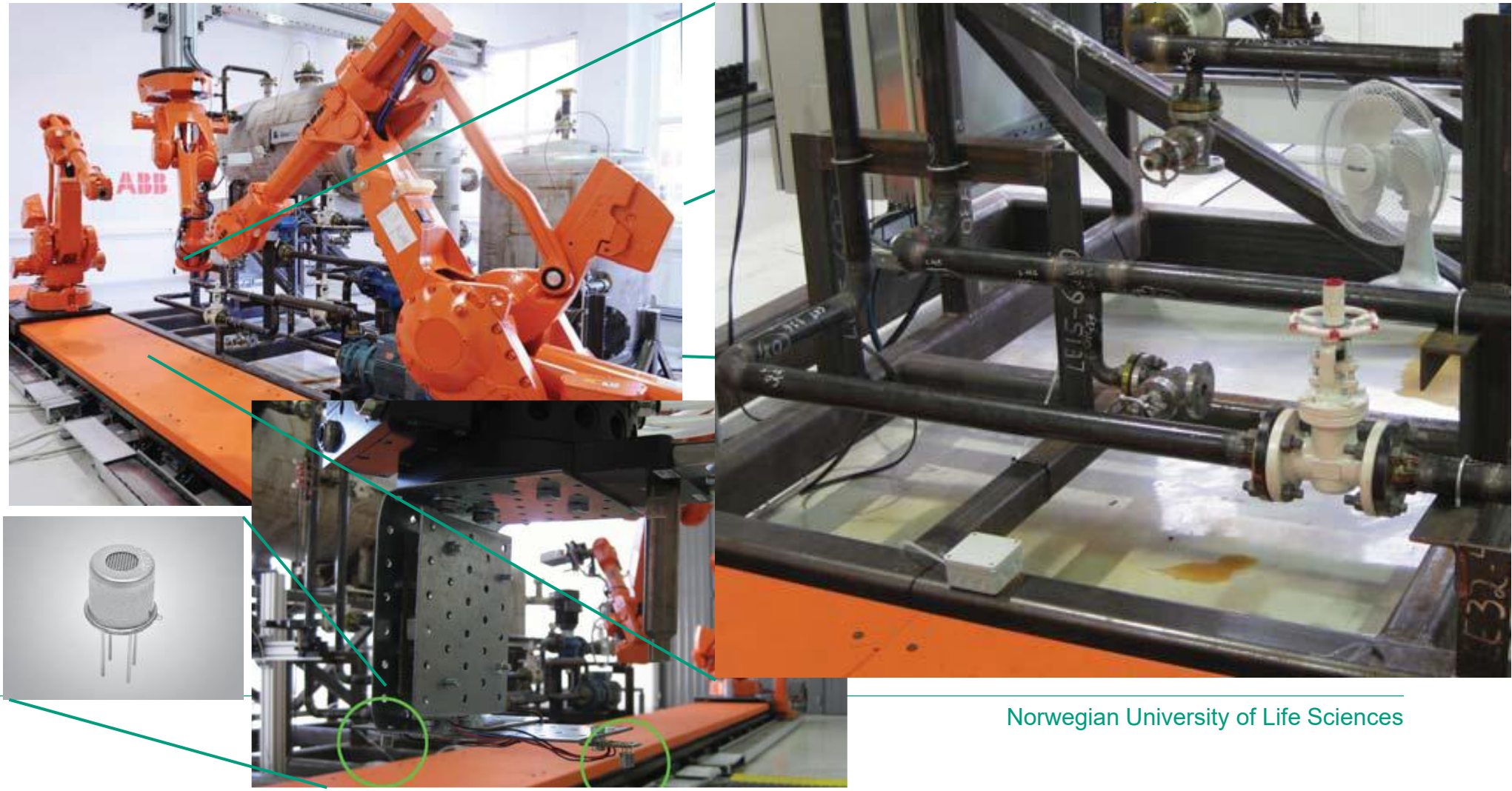
>>> %run -m braitenberg -H

Velocity: $V = 2 - SR - SL$
Steering: $\gamma = k(S_R - SL)$



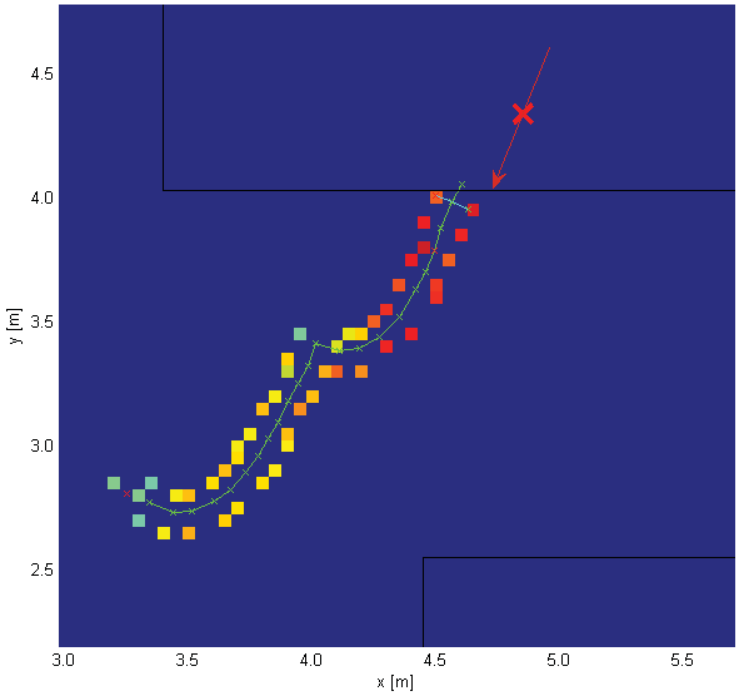


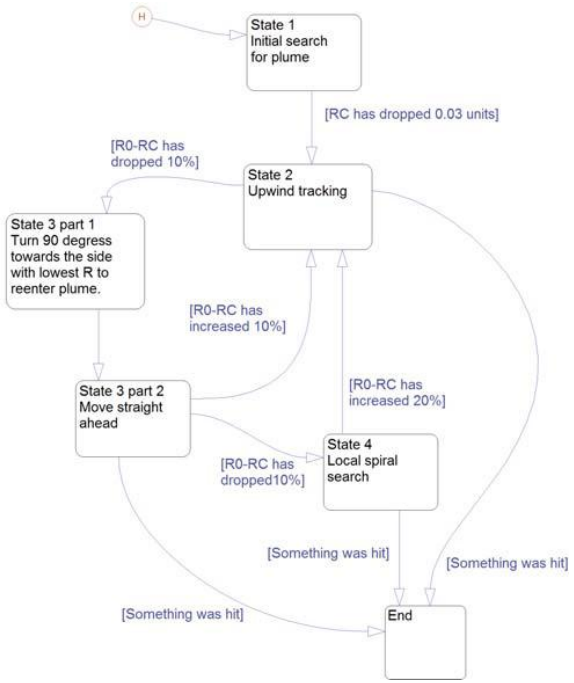
Robotic: gas source localization



Figaro TGS 2620
gas sensor

Robotic: gas source localization





Proceedings of the 18th World Congress
The International Federation of Automatic Control
Milano (Italy) August 28 - September 2, 2011

A Comparative study of robotic gas source localization algorithms in industrial environments ⋆

Erik Persson * David A. Anisi *

* *Department for Strategic R&D for Oil, Gas & Petrochemicals, Process Automation Division, ABB, Oslo, Norway*
e-mail: {erik.persson, david.anisi}@no.abb.com

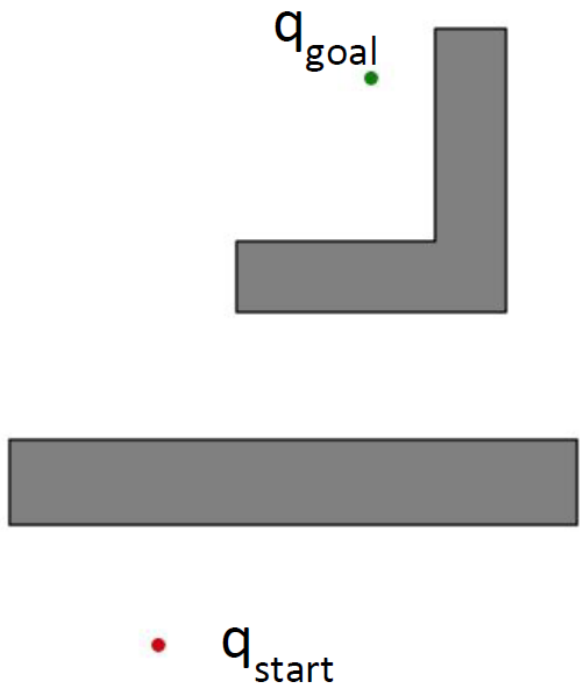
Abstract: Gas source localization is the concept of locating the source of a chemical substance spreading in the environment. Within the oil, gas and petrochemical industry, there is an extreme focus on health, safety and environment (HSE) issues. Hence, being able to locate the source of a gas leakage in an accurate, reliable and quick manner, is of greatest interest to the industry. Although robotic gas source localization has been an active field of research for over fifteen years, large scale real world applications are still lacking. Towards closing this gap, this work presents the results of a comparative study between five different algorithms for robotic gas source localization. The first three are taken from the literature, while the last two are novel modifications and combinations of them. In addition to describing these algorithms, this paper presents the details of the conducted comparative study and the the results thereof.

Keywords: Industrial robots, Gas source localization, Autonomous mobile robots, Control algorithms.

Rodney Brooks: behavior-based, subsumption architecture

Simple automata: bug algorithms

- Assumptions on the robot:
  - Operate in a static grid world
  - Occupies one grid cell
  - Omnidirectional
  - Has only local sensing capability/ information (proximity sensor), inkl. detection of obstacles in adjacent cells
  - Able to self-localize
  - Has knowledge about a global goal (position and orientation/direction)

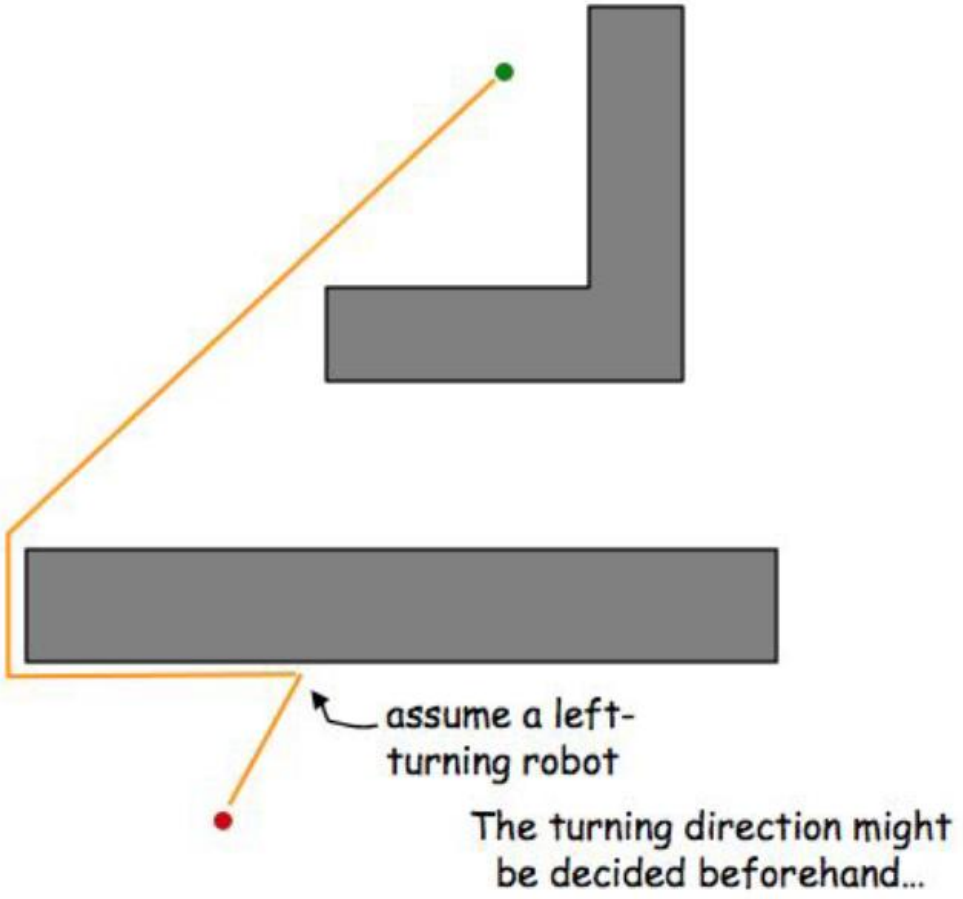


Bug0

- *Bug Algorithmic Behavior*
  - Follow a wall (take a decision on Right – Left)
  - Move towards the goal

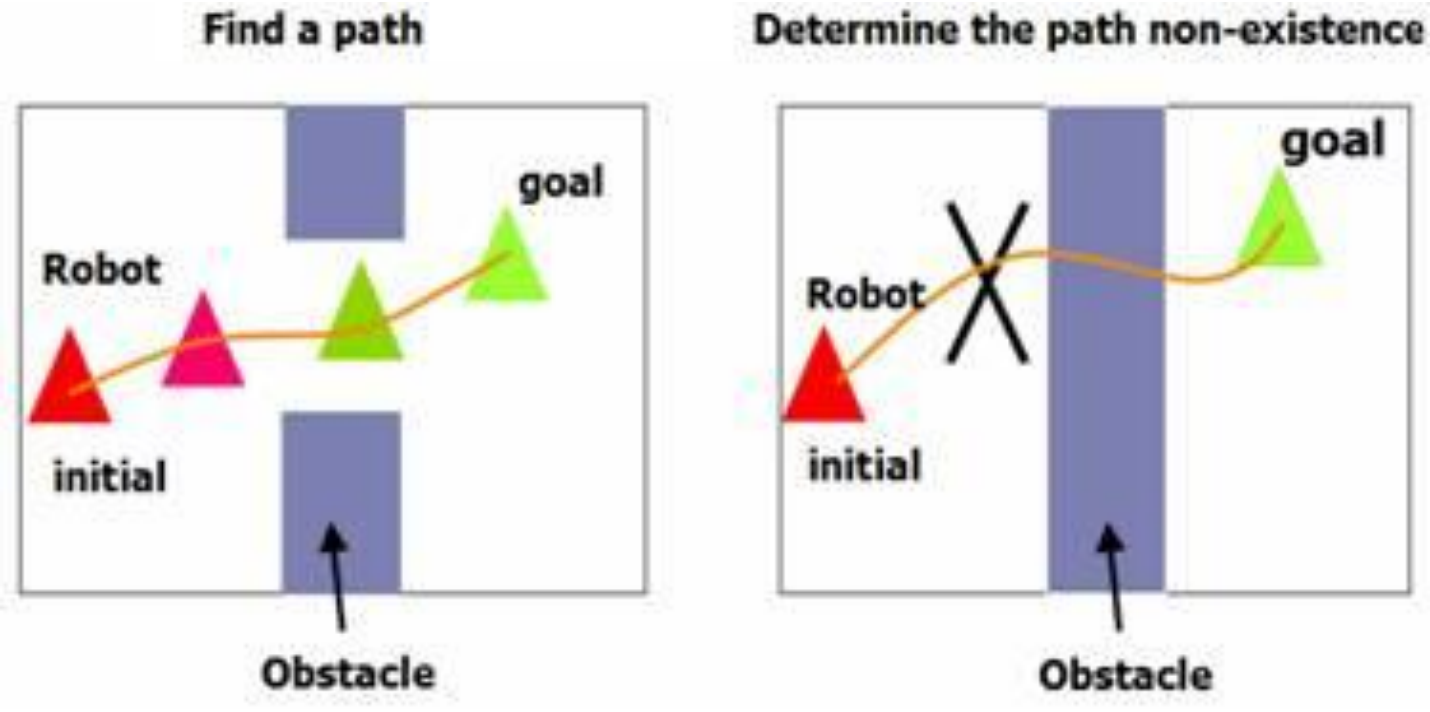
<u>Bug0 algorithm:</u>

1. Head towards goal
2. Follow obstacles until you can head towards the goal again
3. Repeat



Completeness of path planning algorithms

- Definition (Completeness): A path planner is *complete* if, in finite time, it finds a path if such a path exists, or terminates with failure if it does not.

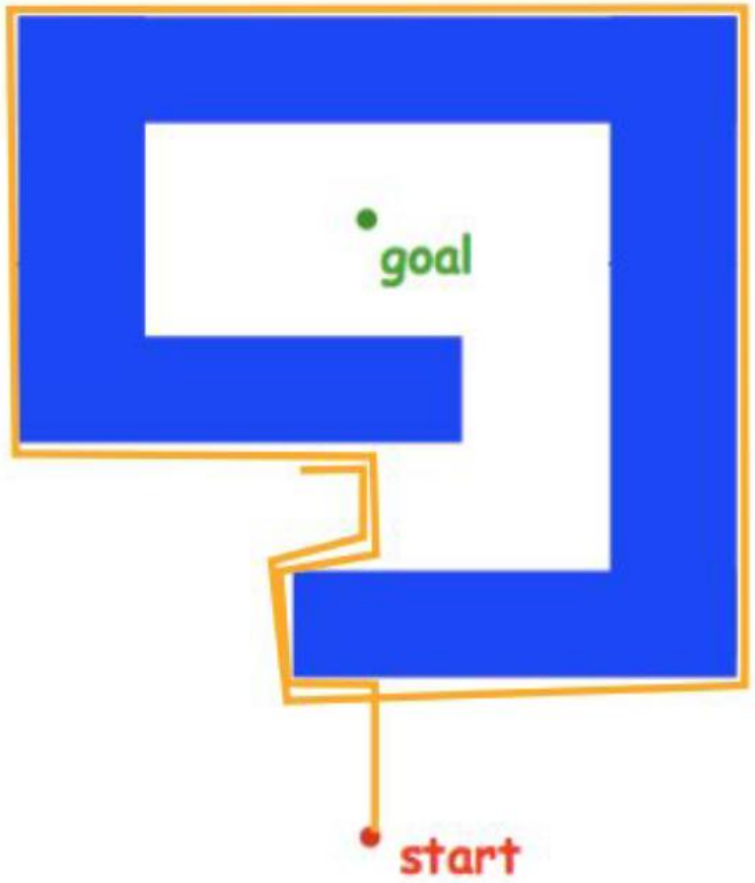


Student group activity: Bug0

Sketch an environment (obstacle configuration) where Bug0 algorithm fails to:

1. find a path, although one exists
2. terminate gracefully and report non-existence of feasible path

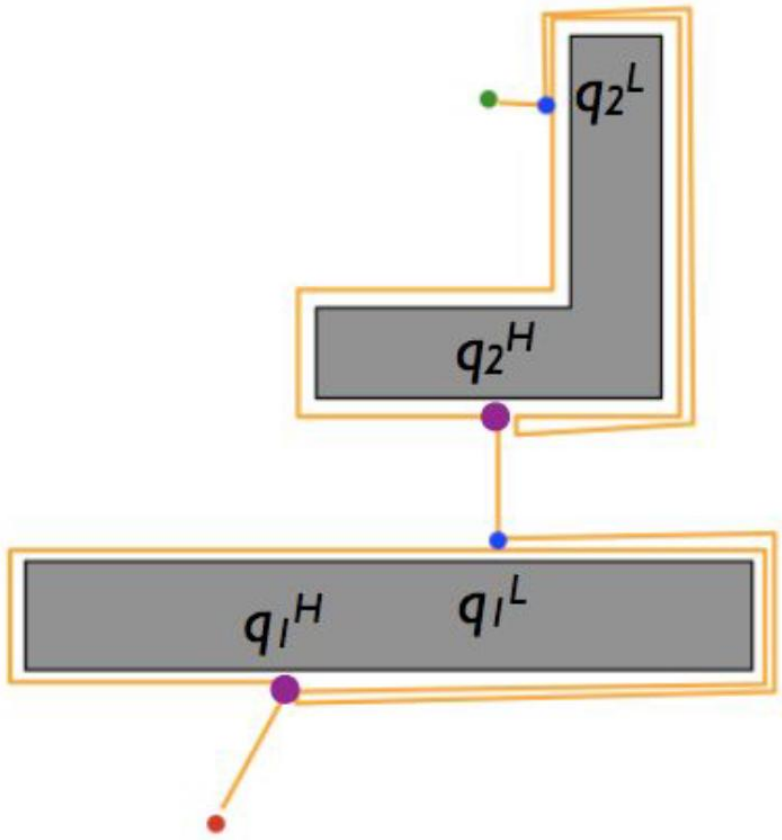
Bug0



Bug1

<u>Bug1 algorithm:</u>

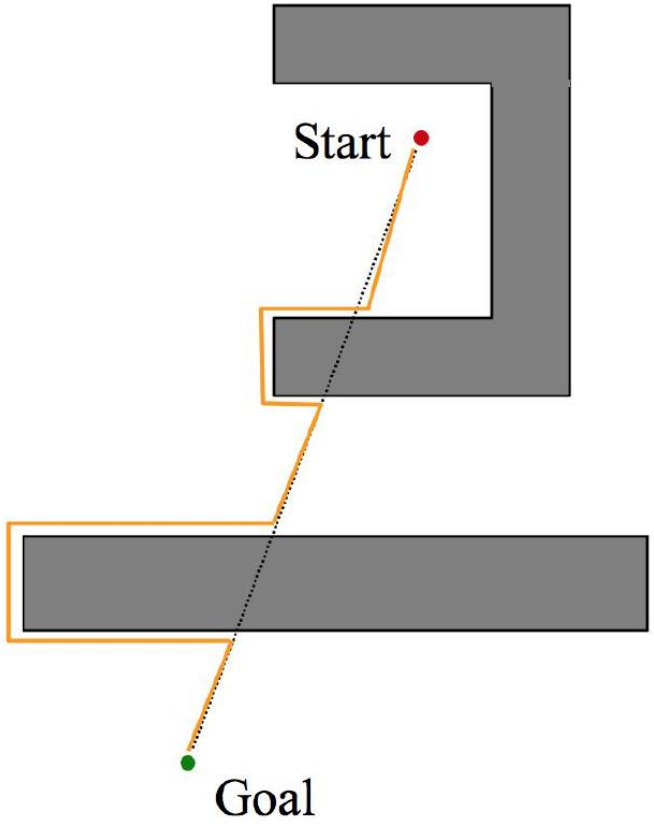
1. Head towards goal
2. If an obstacle is encountered circumnavigate it and remember how close you get to the goal
3. Return to the closest point to the goal and repeat



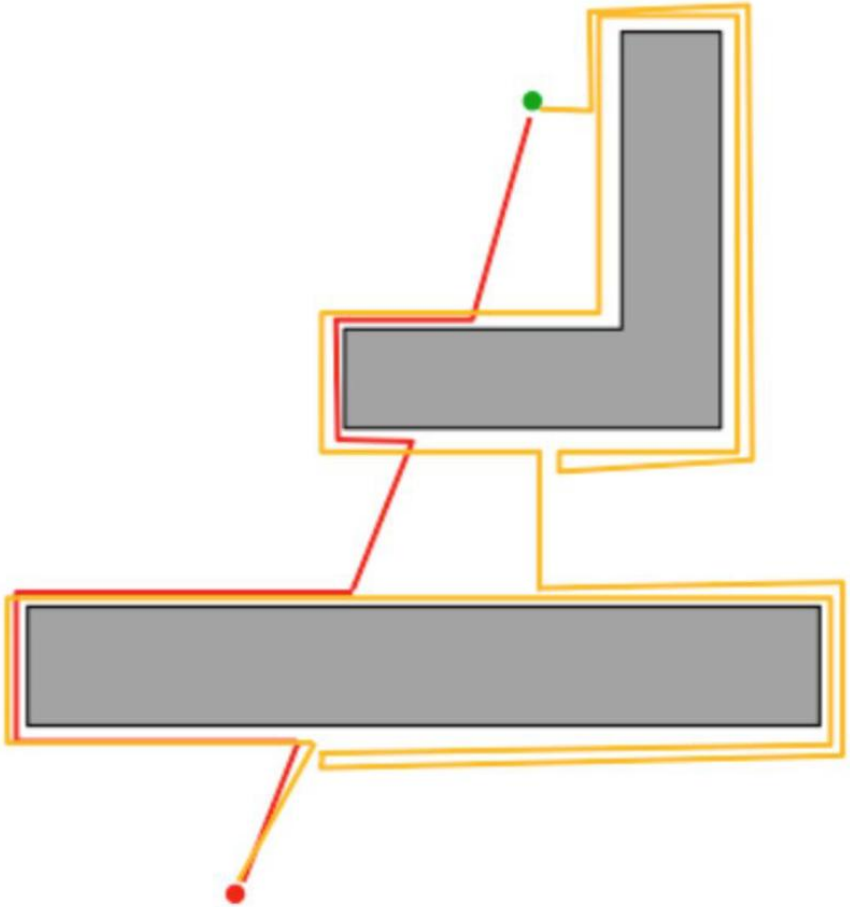
Bug2

<u>Bug2 algorithm:</u>

1. Head towards goal on the m-line
2. If an obstacle is in the way, follow it until you encounter the m-line again closer to the goal
3. Leave the obstacle and continue towards the goal

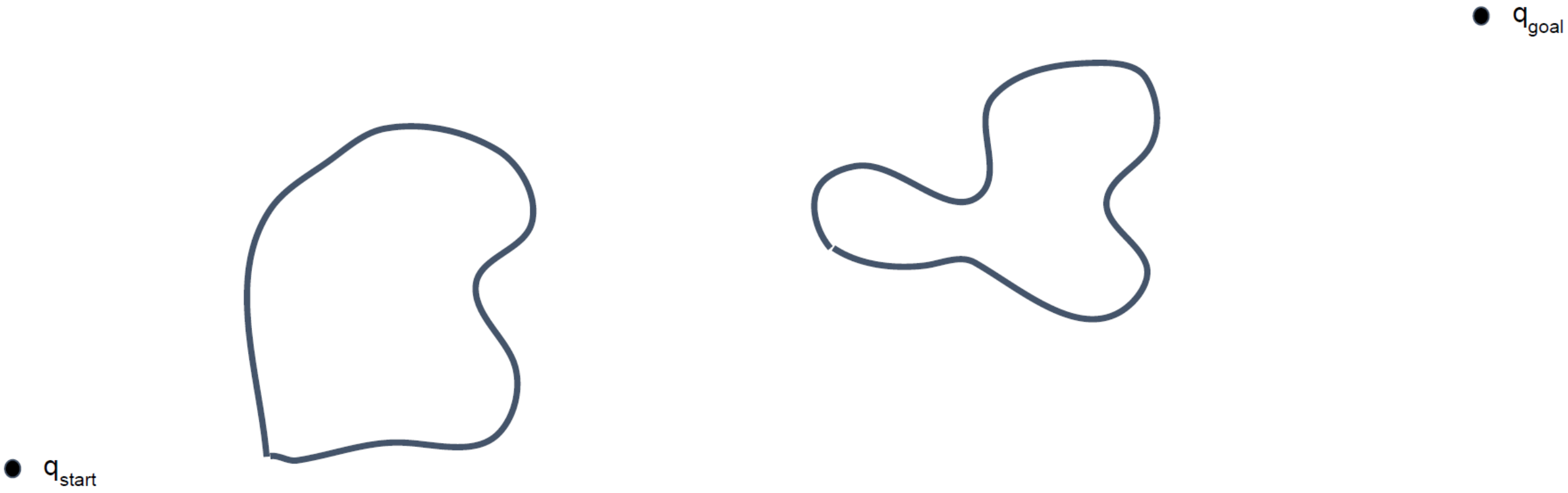


Bug1 and Bug2 comparison

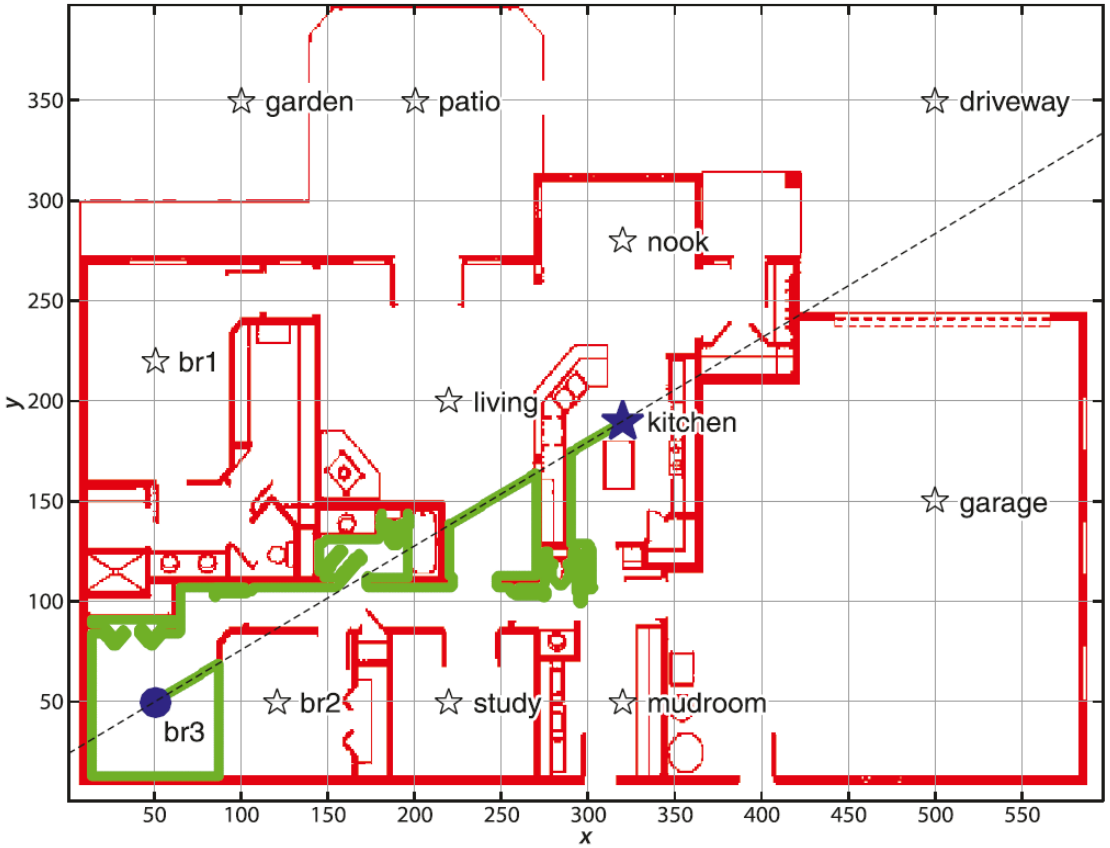


Student group activity: comparison of Bug algorithms

- Sketch and compare the paths generated by Bug0,1,2 algs. on this example:



Bug2 Path-planning

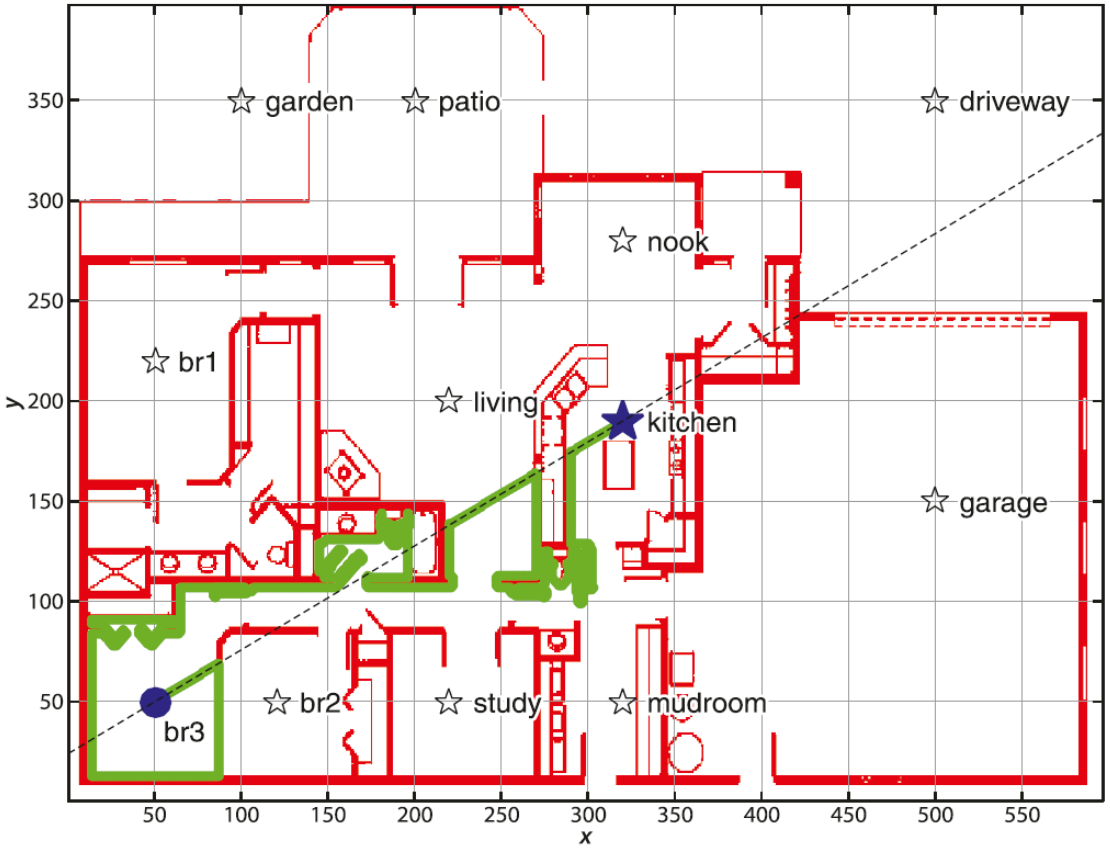



```
>>> house = rtb_load_matfile("data/house.mat");

>>> floorplan = house["floorplan"];

>>> places = house["places"];

>>> bug = Bug2(occgrid=floorplan);

>>> bug.run(start=places.br3, goal=places.kitchen, animate=True);

>>> path = bug.run(start=places.br3, goal=places.kitchen);
```

Bug2 Path-planning

Observations and comments:

- Found a (very) suboptimal route
- Would a slightly different strategy, e.g. turning left, give different outcome?
- Smoke-diver's dilemma
- Performance fundamentally limited by local info/sensing
- Use of world-map is the key for finding «best» (e.g. shortest) paths (next lecture)



Thank you for your attention!